



Pre-launch readiness and go-live

How a finished site on its `netlify.app` address becomes the client's live site on their own domain — audited, keyed, paid-path-tested, signed off and cut over. If you remember one thing: **nothing is verified until you have seen it on the live domain**, and nothing that touches money is done outside Claude Fable against a test key.

SOP ID	HGM-SOP-WEB-005	EFFECTIVE	2026-09-29
VERSION	0.2	NEXT REVIEW	2027-03-29
OWNER	Web & Content Specialist (Kyle Zinger)	TRIGGER	Once per site, before go-live
APPROVED BY	Operations Manager (Gillian) — pending	DURATION	Unconfirmed — record it on the first run

01 PURPOSE AND SCOPE

Purpose

Takes a site that WEB-004 left complete on its `*.netlify.app` address and makes it ready to be a business: search and answer-engine ready, accessible to WCAG 2.2 AA, fast on every page, correct at every screen size, legally complete, wired to the client's live payment, email, booking and CRM accounts, proven with a test booking, approved by the client, and moved onto the client's real domain without taking their email down.

In scope

SEO and GEO. Accessibility to WCAG 2.2 AA. The Lighthouse loop. BrowserStack responsive checks. Legal and policy pages. The HGM and Meta client pixels. Collecting and installing the live keys — Stripe, Resend, the booking platform, GoHighLevel. The money path. A test booking and the booking-flow walk. The Resend check. Client sign-off. The DNS cutover. Confirming the site is live.

Out of scope

Everything WEB-004 covers — the build, the Resources folder, the forms, matching the original. Google Analytics 4, Search Console and Hotjar, which are WEB-003 and start the day the site is live.

This SOP starts at WEB-004 task 8.4: the pull request is merged, the team has reviewed the site in the client websites space, and the Asana task lists the keys still missing. It ends where WEB-003 phase 1 begins: the site answers on the client's domain and the repository can be proven to be the live site.

 This is a field-test draft

Version 0.1 was built before the interview answers, the WCAG audit prompt and the promo-code campaign links document arrived. Every gap is marked **[NEEDS INPUT]** in place. Run the first launch against it, write down what actually happened at each flag, and hand the answers back — v0.2 clears them. Do not treat a flagged task as an instruction.

ALWAYS TRUE

Six standing rules

These hold on every launch. Everything else in this document is procedure; these are not negotiable.

1

Money work happens in Claude Fable, against test keys, never a real charge

Anything that touches prices, coupons, the payment intent, checkout, webhooks or an API key is done in Claude Fable with Stripe in test mode. A live secret key never sits in `.env.local` on your machine and never appears in a prompt.

2

The client's email must survive the cutover

Before any DNS record changes, every existing MX, SPF, DKIM and DMARC record is written down; after the change a test message is sent to the client's own address and seen to arrive. A launch that breaks the client's inbox is a failed launch.

3

Nothing is verified until it has been seen on the live domain

A check passed on `netlify.app`, on a deploy preview or on `localhost` is a check that has to be repeated on `https://<client-domain>` after cutover. Phase 11 exists for this.

4

Every audit prompt's output is checked, not trusted

Claude reports "all pages 100/100" or "the site is AA"; you open the page and look. Each audit phase pairs the prompt with a by-hand check, and the by-hand check is the one that counts.

5

Never invent a value

A key, a Location ID, a pixel ID, a domain record — it comes from the client's record in `hgportal.com`, from 1Password, or from the client through their account manager. If it is not there, it is a named gap on the Asana task, never something typed in from another client.

6

The client approves before DNS moves

Written sign-off is recorded on the Asana task before phase 10 begins. No sign-off, no cutover, whatever the deadline says.

ROLE	RESPONSIBILITY
Web team member running pre-launch	Performs tasks 1.1 to 11.4. Anyone on the web team can run this SOP. The cutover phase is run with a second person watching the first time.
Web & Content Specialist (Kyle Zinger)	Owns this SOP and keeps it current. First point of contact when the procedure and reality disagree.
Template owner (Leshan Patterson)	Owns the two template repositories, the webhook endpoints they expose and the environment variable names. Escalation when the template's money path or key handling is wrong.
Account manager for the client	Requests from the client everything the client has to supply — Stripe access, policy wording, DNS access or a DNS change, the sign-off — and carries the cutover window back.
Client	Owns the domain, Stripe, Netlify, Supabase and the booking platform account. Supplies keys and wording, approves the site, agrees the cutover moment.
Operations Manager (Gillian)	Approves this SOP. [NEEDS INPUT: Q27 — whether Gillian also signs off each launch, or only the client]
Client websites space, Google Chat	Where the site is posted for team review before client sign-off, and where a cutover is announced as it happens.

1 The merged WEB-004 build

On main, deployed to its `netlify.app` address, with the Asana task in the pre-launch stage and its comment listing the keys still missing.

2 A login for hgmportal.com

The client record holds the property management system and the keys HGM already has. **[NEEDS INPUT: Q1 — whether the registrar and DNS host are recorded there]**

3 1Password, with the client's vault

The site's `.env.local` entry and the client's logins. See HGM-SOP-WEB-001.

4 VS Code with Claude Code signed in

Claude Fable selected for phases 7 and 8. **[NEEDS INPUT: how Claude Fable is selected in Claude Code at HGM — the model picker in the prompt box, or a setting]**

5 The client's Netlify team

With the HGM web team account as a Developer (WEB-004 task 2.1).

6 The client's Stripe account in test mode

Or sandbox details, supplied by the client together with a 100% coupon (task 6.2).

7 The client's GoHighLevel sub-account

The same one the forms were built in (WEB-004 task 7.4).

8 A BrowserStack login

[NEEDS INPUT: Q22 — whether HGM has an account, and whether Claude reaches it through an MCP or you check by hand]

9 Terminal tools and a phone

`git`, `dig`, `curl`, `npx`, a browser with developer tools, and a real phone for task 4.4.

10 The Paradise Pointe site as the reference

For SEO wording and the legal pages. **[NEEDS INPUT: Q23 — the Paradise Pointe URL, and what "the master doc" is and where it lives]**

Where are you working?

VS Code

Every audit prompt, every fix, every commit. Claude Code runs here, and Claude Fable for phases 7 and 8.

Terminal

The `dig` and `curl` checks in phases 10 and 11. Everything here also works from the VS Code terminal.

Claude Desktop app

Prompts only. Everything that is a key, a record or a dashboard — Stripe, Resend, GoHighLevel, Netlify, the DNS host — is point and click in a browser.

Pick one route per task

Wherever a task can be done more than one way you will see a point-and-click box, a terminal box or a Claude prompt box. They do the same thing. Only routes that exist are offered; a task with one box has one route.

GEO (generative engine optimisation)

Making a site legible to AI answer engines the way SEO makes it legible to Google: structured data that says what each page is, and an `llms.txt` file at the site root describing the site in plain text.

Schema (structured data)

JSON-LD blocks in a page's HTML describing the page's entity — a vacation rental, an organisation, an FAQ — in a vocabulary search and answer engines read. Checked with Google's Rich Results Test.

WCAG 2.2 AA

The Web Content Accessibility Guidelines at level AA, the standard phase 3 audits to: keyboard operability, text alternatives on images, colour contrast, visible focus, and the rest.

**[NEEDS
INPUT:
WCAG audit
prompt —
the criteria
list it
enumerates]**

Lighthouse

Google's page audit tool, scoring Performance, Accessibility, Best Practices and SEO out of 100. The loop in phase 4 runs it until every page scores 100 in every category.

Test mode (Stripe)

Stripe's sandbox. Test keys begin `pk_test_` and `sk_test_`; live keys begin `pk_live_` and `sk_live_`. A charge made with a test key is not real. Every money task in this SOP happens in test mode.

Webhook

A URL on the site that Stripe or the booking platform calls when something happens — a payment succeeds, a reservation is created. It is registered on their side with the site's address, and it has to be re-registered when that address changes at cutover.

What are you trying to do?

Pick the phase that matches your situation and jump straight to its tasks.

PHASE 1

Confirm the build is ready to be checked

Tasks 1.1–1.4 · start of pre-launch, once

PHASE 2

SEO and GEO

Tasks 2.1–2.5 · before any audit reads the pages

PHASE 3

Accessibility to WCAG 2.2 AA

Tasks 3.1–3.4 · after SEO, before performance

PHASE 4

Performance and responsive checks

Tasks 4.1–4.4 · Lighthouse loop, then BrowserStack

PHASE 5

Legal and policy pages

Tasks 5.1–5.4 · needs wording from the client

PHASE 6

Collect and install the live keys

Tasks 6.1–6.10 · the list from WEB-004 task 8.4

PHASE 7

The money path, in Claude Fable

Tasks 7.1–7.5 · test keys only, never a real charge

PHASE 8

Test booking and the full flow

Tasks 8.1–8.6 · one real booking, cancelled after

PHASE 9

Client sign-off

Tasks 9.1–9.3 · written approval before DNS moves

PHASE 10

DNS cutover

Tasks 10.1–10.11 · the point of no return

PHASE 11

Confirm live and hand to WEB-003

Tasks 11.1–11.4 · everything again, on the real domain

Phase 1 — Confirm the build is ready to be checked

Prove you are starting from the merged build, on a machine whose environment is filled, on a branch you can throw away.

1.1 Read the handoff on the Asana task

Open the Asana task and read the comment WEB-004 task 8.4 left: the `netlify.app` address, the list of keys still missing with a name against each, and the `TOKENS.md` total.

Expected result

You know which keys phase 6 has to collect before you open a single dashboard.

1.2 Pull main and confirm the deployed site is main

Fetch, fast-forward, then compare the commit Netlify has deployed with `origin/main`.

 **TERMINAL /**
CLI

```
git fetch origin && git status -sb
git pull --ff-only
git rev-parse origin/main    # compare with the commit shown on the Netlify deploy
```

 **CLAUDE**
PROMPT

PASTE INTO CLAUDE CODE

Fetch origin and tell me how far behind main is. If a fast-forward is possible, pull. Then ask the Netlify connector for `get-deploy-for-site` on this site and tell me whether `commit_ref` equals `origin/main`. Make no changes.

Expected result

`git status -sb` shows `main...origin/main` with no `[behind N]`, and the deployed commit equals `origin/main`.

1.3 Check `.env.local` is filled, by length

Print each key with the length of its value, never the value. If keys are blank, restore the file from the client's 1Password entry (HGM-SOP-WEB-001) and restart the dev server.



TERMINAL /
CLI

LENGTHS ONLY

```
awk -F= '/^[A-Za-z_][A-Za-z0-9_]*=/ { print $1, length($2) }' .env.local
```



CLAUDE
PROMPT

PASTE INTO CLAUDE CODE

Check .env.local: list every key with the LENGTH of its value. Never print a value. Tell me which keys are blank.



Never print or paste a value

Lengths, not contents. A value pasted into a chat, an Asana comment or a screenshot is a key that has to be rotated.



Expected result

Every key WEB-004 filled shows a non-zero length, and the blank ones are exactly the list from task 1.1.

1.4 Make the pre-launch branch and open the cost row

Create a branch named pre-launch off main following HGM-SOP-WEB-002, and add today's row to TOKENS.md (WEB-004 task 3.4).



TERMINAL /
CLI

```
git checkout -b pre-launch
```



Expected result

Every audit fix in phases 2 to 5 lands on pre-launch and reaches production through one pull request in task 8.6.

Phase 2 — SEO and GEO

Give every page its titles, descriptions and social shares, then make the site legible to answer engines. The reference for voice and tone is the finished Paradise Pointe site.

2.1 Set meta titles, descriptions and social shares for every page

In Claude Code, run the prompt below with the Paradise Pointe URL and the master doc attached. **[NEEDS INPUT: Q23 — the Paradise Pointe URL and the master doc's name and location]**

 **CLAUDE
PROMPT**

FILL IN THE TWO ANGLE-BRACKET VALUES

Set the meta title, meta description and social share (Open Graph and Twitter card) for every page on this site. Use <Paradise Pointe URL> as the reference for a site that is already done, and read <master doc> to understand the voice and tone to write in. List every page and the values you set.

Expected result

A list of every page with its title, description and share image. Compare the list against the sitemap; no page may be missing.

2.2 Check the tags on three pages yourself

Open the home page, the listings page and one property page on the deploy preview. View source and find <title>, <meta name="description", og:title, og:description and og:image on each.

 **POINT &
CLICK**

THEN REPEAT FOR DESCRIPTION

Browser: View Page Source (Cmd + Option + U in Chrome) → Cmd + F "og:image".

 **TERMINAL /
CLI**

REPLACE <DEPLOY-PREVIEW>

```
curl -s https://<deploy-preview>/ | grep -o '<meta property="og:[a-z:]*" content="[^"]*"'
```

Image to be added

(image to be added) — One page's rendered <head> showing the title, description and social tags.

👁 Expected result

Every tag is present and the share image URL loads. A title that reads as a template placeholder is a fail.

2.3 Run the GEO audit — schema and llms.txt

In Claude Code, run the prompt below.



PASTE INTO CLAUDE CODE

```
Do a full GEO audit of this site. Optimise the site for generative engines: add schema where it is missing so that each page's entity is described, update existing schema so LLMs know what each page is about, and create an llms.txt file at the site root describing the site, its pages and its properties. Fill schema only from data the site actually has; leave out anything you would have to invent. List every change.
```

👁 Expected result

JSON-LD on every page type, one builder function per entity (WEB-003 task 7.2 explains why), and llms.txt at the site root.

2.4 Check the schema and read llms.txt

Run Google's Rich Results Test on the listings page first, then one property page. Open <https://<deploy-preview>/llms.txt> and read it.

i Test the list page, not just the detail page

On Ridge & Falls the listings page emitted a thin copy of each property with the same @id as the full node on the property page, and the test scored every cabin "2 critical". The list page is where it shows.

👁 Expected result

Zero critical issues on both pages, and llms.txt describes this client and no other.

2.5 Commit and push

Commit the SEO and GEO changes on pre-launch and push. Netlify rebuilds the deploy preview.



TERMINAL /
CLI

```
git add -A && git commit -m "feat: SEO meta, schema and llms.txt" && git push -u origin pre-launch
```

Expected result

The preview carries the new tags and task 2.2's checks pass against it.

Phase 3 — Accessibility to WCAG 2.2 AA

Run the audit prompt, answer its questions, then check the site by hand against the criteria.

3.1 Run the accessibility audit prompt

In Claude Code, paste the prompt exactly as written.



CLAUDE
PROMPT

PASTE INTO CLAUDE CODE, VERBATIM

Do a quick accessibility check as in an audit on the website and correct all the accessibility issues. Ask questions if you are unsure. Ensure the website is keyboard navigable, AA rated where possible, add alt tags to all images and anything else that needs to be done to enhance accessibility.

Expected result

A list of issues found and fixed, and a list of questions.

3.2 Answer every question before letting it continue

The prompt says "ask questions if you are unsure" — read them. A typical one is what a decorative image's alt text should be, or whether a client colour that fails contrast may be changed.



Keep the client's swatches even when contrast fails

Do not let the audit silently change a brand colour. Document the contrast ratio at the declaration and tell the account manager the passing alternative, so the client decides. WEB-003 section 06 records the same rule.

Expected result

No question is left unanswered, and no brand colour has been changed without the client's say.

3.3 Check keyboard navigation and alt text yourself

On the deploy preview, put the mouse down. Tab from the top of the home page through the search widget, into the listings, onto a property page and into the booking form. Then confirm no image on the listings page lacks an alt attribute.



TERMINAL /
CLI

PRINTS IMAGES WITH NO ALT

```
curl -s https://<deploy-preview>/listings | grep -o '<img[^>]*>' | grep -v 'alt='
```



POINT &
CLICK

Browser: click in the address bar, then press Tab repeatedly.
Watch for the focus ring on every link, button and field.
Press Enter on the search button; keep tabbing into the results.



Image to be added

(image to be added) — The site with a keyboard focus ring visible on a button, proving the indicator is not clipped. Ring the focus ring.



Expected result

You can reach and operate every control with the keyboard, focus is visible at every step, and the grep prints nothing.

3.4 Check the remaining WCAG 2.2 AA criteria

[NEEDS INPUT: WCAG audit prompt — the criteria list, the check for each, and the pass condition. This task becomes a table once the audit prompt is supplied.]



Expected result

Every criterion in the table passes or has a named exception on the Asana task.

Phase 4 — Performance and responsive checks

Two loops, each followed by a look.

4.1 Run the Lighthouse loop

In Claude Code, run the prompt below.

[NEEDS INPUT: Q20 and Q21 — whether the loop runs against localhost, the deploy preview or production; Lighthouse via the CLI, DevTools or a Netlify plugin; whether 100/100 means all four categories; and the accepted floor if the loop cannot get there]



PASTE INTO CLAUDE CODE, VERBATIM

Next create a loop that checks Lighthouse and continues to loop until everything and every page is 100/100.

A loop is blunt here too

WEB-004 warns that a matching loop stops improving while still spending tokens. A Lighthouse loop is safer because the score is a hard number, but if it has run for **[NEEDS INPUT: Q21 — a time or pass limit]** without reaching 100, stop it and fix the remaining items by name.

Image to be added

(image to be added) — A Lighthouse run on the home page showing the four scores at the end of the loop. Ring the Performance score.

Expected result

A per-page table of the four scores, all 100.

4.2 Run Lighthouse yourself on three pages

In Chrome, open DevTools, choose the Lighthouse panel, and run the home page, the listings page and one property page — mobile and desktop.

Screenshot 4.2 to capture

A Lighthouse run showing the four 100s. Crop to the four score circles and the page URL; ring the Performance score.

👁 Expected result

The scores you see match the table Claude reported. If they do not, the loop measured a different build — see section 07.

4.3 Run the responsive check on BrowserStack

In Claude Code, run the prompt below. [\[NEEDS INPUT: Q22 — how Claude reaches BrowserStack \(an MCP, or you drive it\), and the device list HGM tests against\]](#)

CLAUDE
PROMPT

PASTE INTO CLAUDE CODE, VERBATIM

Check mobile responsiveness on BrowserStack, all breakpoints on the website and all device sizes should be perfect.

📘 Image to be added

(image to be added) — The BrowserStack device grid part-way through a responsive pass.

👁 Expected result

A report per breakpoint and device, with any layout fault fixed.

4.4 Look at it on a real phone

Open the deploy preview on your own phone. Walk home, listings, property, the booking form. Open the mobile menu and the pop-up.

👁 Expected result

Nothing overflows, nothing is cut off, the sticky search widget behaves, and the pop-up fits the screen. BrowserStack emulates; a phone is the truth.

Phase 5 — Legal and policy pages

The cancellation policy has to read the same everywhere; the privacy policy and terms of use have to exist and be reachable.

5.1 Collect the wording

Get from the account manager: the cancellation policy, the privacy policy, and the terms of use PDF. **[NEEDS INPUT: Q23 and Q24 — who supplies each: the client, or HGM's template wording modelled on Paradise Pointe; and whose PDF the terms of use is]**

👁 Expected result

Three pieces of text, or a named owner and date on the Asana task for each one missing.

5.2 Build the pages to match Paradise Pointe

Attach the wording and the PDF and run the prompt below.



CLAUDE
PROMPT

FILL IN THE PARADISE POINTE URL

Add a privacy policy page and a terms of use page, matching the layout and placement of the same pages on <Paradise Pointe URL>. Use the attached wording verbatim. The terms of use are the attached PDF – link to it rather than retyping it. Add both to the footer.

👁 Expected result

Both pages are reachable from the footer on every page, and the PDF opens.

5.3 Make the cancellation policy identical everywhere

The policy appears on the property page, on the checkout and confirmation pages and in the confirmation email. Ask for one source of truth and a check.



CLAUDE
PROMPT

PASTE INTO CLAUDE CODE

The cancellation policy must be identical on the property page, the checkout and confirmation pages, and in every email the site sends. Move it to one constant and render it from there everywhere. Then list every place it appears.



TERMINAL /
CLI

EXPECT ONE FILE

```
grep -rl "<a distinctive phrase from the policy>" app lib components
```



Expected result

One definition. A grep for a distinctive phrase from the policy finds it in that one file only.

5.4 Privacy pop-up

[NEEDS INPUT: Q24 — whether the template ships a privacy or cookie pop-up and whether it is required; the notebook line is struck through]



Expected result

The decision is recorded on the Asana task, and the pop-up is either present on every page or deliberately absent.

Phase 6 — Collect and install the live keys

Work through the missing-keys list from task 1.1. Every value goes into `.env.local`, then onto Netlify by hand, then back to 1Password.

6.1 Open the list and ENV.template side by side

Match each missing key to its variable name. [NEEDS INPUT: Q18 — the variable names from ENV.template for Stripe, Resend, Meta, GHL and the booking platform, names only. Nothing below names a variable until this is supplied.]



Expected result

Every missing key has a variable name and a source.

6.2 Get Stripe test access and the 100% coupon from the client

Through the account manager, ask the client for either the test publishable key and test secret key from their Stripe account, or sandbox details, plus a 100% coupon for the test booking. [NEEDS INPUT: Q11 — whether Stripe is the client's own account, whether HGM ever holds a live secret key, and who swaps test for live] [NEEDS INPUT: Q12 — whether the 100% coupon is created in Stripe or in the site's own coupon store]



MESSAGE FOR THE ACCOUNT MANAGER TO
FORWARD

For <client>'s launch we need Stripe test access so we can prove the payment path without charging anyone: the test publishable key and test secret key from their Stripe dashboard (test mode), or sandbox login details, plus a 100% coupon code we can use for one test booking. Please send them through 1Password, not email.

⊗ Test keys only, on this SOP

A key beginning `sk_live_` does not go into `.env.local` on anyone's machine. If the client sends live keys, put them in 1Password and stop; the live swap happens in task 11.3.

ⓘ Screenshot 6.2 to capture

Stripe in test mode, the API keys page. Crop to the two key rows with the values hidden; ring the test-mode toggle.

ⓘ Image to be added

(image to be added) — Stripe's test-mode API keys screen, both values redacted before you send it. Ring the test-mode toggle.

👁 Expected result

Two keys beginning `pk_test_` and `sk_test_`, and a coupon code, in the client's 1Password vault.

6.3 Get the Resend API key

[NEEDS INPUT: Q13 — whether Resend is the client's account or HGM's, where the key is created, and what the notebook's "Netlify → user → settings → OAuth → new access token" line was for. That path is Netlify's personal access token screen, not Resend.]

ⓘ Image to be added

(image to be added) — The Resend API keys screen, value redacted.

👁 Expected result

A Resend API key in 1Password, and the sending domain it is tied to written on the Asana task.

6.4 Get the Meta values

The CAPI access token is one HGM value shared across all clients; take it from **[NEEDS INPUT: Q19 — where it is held]**. The pixel ID is **[NEEDS INPUT: Q19 — per client from the client's Meta Business account, or shared]**.

Expected result

Both values are in `.env.local` under the names from task 6.1.

6.5 Get the GoHighLevel Location ID and private integration token

In the client's GoHighLevel sub-account, open the business profile and copy the Location ID. Then create a new private integration, select all scopes, and copy its token. **[NEEDS INPUT: Q16 — whether "select all" means literally every scope, and the exact menu labels in HGM's white-labelled GoHighLevel]**

Screenshot 6.5 to capture

The GoHighLevel private integration screen after creation. Crop to the scope list and the token row with the token hidden; ring the create button.

Check which sub-account you are in

A token from the wrong sub-account sends this client's bookings into another client's CRM, and nothing errors. Confirm the Location ID before you create anything.

Image to be added

(image to be added) — The GoHighLevel private integration screen after creation, with the scope list showing. Ring the scope list.

Expected result

The Location ID matches the one on the client's record in hgmportal.com (WEB-004 task 7.4), and the token is in 1Password.

6.6 Get the booking platform's booking engine URL

For Guesty: open Marketing + Sales, then Distribution, then Guesty Booking Engine, and copy the booking engine API (BEAPI) URL. **[NEEDS INPUT: Q17 — which variable the BEAPI URL fills, and the Hostaway equivalent or a statement that Hostaway is out of scope for now]**

AwayFrames is on Hostaway

The first run of this SOP is a Hostaway site. Record what the Hostaway equivalent of this task actually is, so v0.2 can carry both platforms.

Image to be added

(image to be added) — The booking platform's distribution screen where the booking engine URL is shown. Ring the URL field.

Expected result

The URL is in `.env.local`.

6.7 Add the values to `.env.local`, then to Netlify by hand

One at a time, in the Netlify project's environment variables, exactly as WEB-004 task 4.6 does it. Never bulk-import the file.

Add them by hand, one at a time

Bulk-importing `.env.local` uploads everything in the file, including anything personal to your machine, and arms every scheduled function the template ships.

Image to be added

(image to be added) — Netlify **Environment variables** with the full list for this site, values redacted.

Expected result

Every variable from task 6.1 exists on Netlify with a non-empty value.

6.8 Register the webhooks and confirm the keys, in Claude Fable

Switch to Claude Fable and run the keys-and-webhooks prompt. **[NEEDS INPUT: Q14 — what the "Supabase access token" is: a personal access token for the management API, or the service_role key WEB-004 already collected] [NEEDS INPUT: Q15 — the webhook endpoint paths the template exposes for Stripe and for the booking platform, where the Stripe webhook signing secret is set, and whether webhooks are registered against netlify.app now and re-registered on the live domain in task 10.5]**



CLAUDE PROMPT -
FABLE

PASTE INTO CLAUDE CODE WITH FABLE SELECTED

The Stripe publishable and secret test keys have been added. Register the webhooks for both the booking platform and Stripe. The Supabase access token has been added. The Resend API key has been added. Confirm the checkout page is functional, the webhooks are correctly set up, and the Resend, Meta and GHL keys are working. Add everything to the project on Netlify.



"Working" means a request went through

Claude can only confirm a key by making a call with it. Read what it did: a Resend key is confirmed by a sent test email you can see in Resend's log, a GHL key by a contact appearing in the sub-account, a Meta key by a test event. If it says "the key looks valid" without a call, it did not check.



Screenshot 6.8 to capture

The Stripe webhook endpoint dialog in test mode. Crop to the endpoint URL and the events list; ring the add-endpoint button.



Image to be added

(image to be added) — The Stripe webhook endpoint dialog with the event list selected. Ring the **Add endpoint** control.



Expected result

Stripe's test-mode webhook list shows the site's endpoint; the booking platform shows its webhook; a report names each key as working.

6.9 Add the HGM and Meta client pixels to the site

[NEEDS INPUT: Q19 — what the HGM pixel is, and whether the template's env-gated Meta pixel block (WEB-003 task 3.1) covers both, so that this is a variable rather than code]



TERMINAL /
CLI

PRINTS THE PIXEL ID THE PAGE CARRIES

```
curl -s https://<deploy-preview>/ | grep -o "fbq('init', '[0-9]*')"
```

Image to be added

(image to be added) — Meta Events Manager showing test traffic arriving from the site. Ring the live event row.

Expected result

The pixel script is in the live HTML of the deploy preview's home page, with the client's ID.

6.10 Write everything back to 1Password and update the Asana list

Save the current `.env.local` into the client's vault per HGM-SOP-WEB-001, and edit the Asana comment so the missing-keys list is empty or names who still owes what.

Expected result

Nothing is missing that phase 7 needs.

Phase 7 — The money path, in Claude Fable

Everything here is in Claude Fable, in Stripe test mode. A mistake here charges a real guest the wrong amount once the site is live, so this phase is never merged with another and never rushed.

7.1 Confirm you are in Claude Fable and in test mode

Check the model shown in Claude Code is Claude Fable, and check that every Stripe value in `.env.local` begins with a test prefix.



TERMINAL /
CLI

PRINTS THE FIRST EIGHT CHARACTERS ONLY

```
awk -F= '/STRIPE/ { print $1, substr($2,1,8) }' .env.local
```

Expected result

The model is Fable and every Stripe value starts `pk_test_` or `sk_test_`. A live prefix stops this phase.

7.2 Walk the checkout page and the payment intent

Ask Fable to trace the money path end to end and report, without changing anything.



CLAUDE PROMPT –
FABLE

PASTE INTO CLAUDE CODE WITH FABLE SELECTED

Trace the money path from the property page to a confirmed booking: where the price is calculated, where deposits or fees are added, where the coupon is applied, where the Stripe payment intent is created and with what amount and currency, and what the webhook does on payment success. Report it as a numbered list with the file and function for each step. Make no changes.



Expected result

A list you can follow in the code. The amount sent to Stripe is the amount shown to the guest, in the currency the client charges in.

7.3 Set up the coupon

[NEEDS INPUT: promo-code campaign links document — how the 100% coupon and any campaign coupons are created, where they live, and how they are applied]



Expected result

The 100% coupon applies at checkout and the total reads zero.

7.4 Build the campaign links

[NEEDS INPUT: promo-code campaign links document — the link format, which pages accept a code from the URL, and how a code is validated]



Expected result

A campaign link opens the site with its code applied, and an invalid code is refused with a message.

7.5 Prove it in Stripe test mode

Make a test payment with a Stripe test card, then open Stripe in test mode and find the payment and the webhook delivery.

⊗ **Never a real charge**

If at any point a live key, a real card or a live-mode dashboard is in front of you, stop and go back to task 7.1.

i **Image to be added**

(image to be added) — Stripe test mode showing a succeeded payment intent for the test booking, with the discounted total. Ring the amount.

👁 **Expected result**

The payment appears with the right amount, and the webhook endpoint from task 6.8 shows a delivered event for it. A payment with no webhook delivery means the booking was paid for and never recorded.

Phase 8 — Test booking and the full flow

One booking, as a guest would make it, from the search widget to the confirmation email — then cleaned up.

8.1 Walk the booking flow as a guest, with dates and data

Start at the home page search widget. Choose a location, dates and guests; open the listings; open a property; start the booking. At every step confirm the dates, guest count and price carry through unchanged.

👁 **Expected result**

What you chose on the home page is what the checkout shows.

8.2 Make the test booking

Complete the booking on the deploy preview using the 100% coupon from task 6.2. **[NEEDS INPUT: Q25 — or a Stripe test card; a real property and real dates, or a designated test property]**

👁 **Expected result**

The confirmation page, with the booking reference.

8.3 Check the confirmation email in Resend

Open Resend and find the confirmation email in its log. Open the message and read it: the cancellation policy from task 5.3, the dates, the property, the amount.

Screenshot 8.3 to capture

The Resend email log with the confirmation delivered. Crop to the row; ring the delivered status.

Image to be added

(image to be added) — The Resend log showing the confirmation email delivered, recipient redacted. Ring the delivered status.

Expected result

The email was delivered, not just sent, and its content matches the confirmation page. Resend shows an error here if anything in the sending path is wrong.

8.4 Check the reservation reached the booking platform, then cancel it

Open the client's Guesty or Hostaway account and find the reservation. Cancel it. [\[NEEDS INPUT: Q25 — whether the test booking creates a real reservation and who cancels it\]](#)

Expected result

The reservation existed, and the dates are free again.

8.5 Check the downstream systems

Confirm the guest appeared as a contact in the client's GoHighLevel sub-account, and that the Meta event fired. [\[NEEDS INPUT: how HGM checks a Meta test event — Events Manager test events, or not checked at this stage\]](#)

Expected result

One new contact, one purchase event.

8.6 Do the manual check, then merge

Open every page in the sitemap, one after another, on desktop, and look at each one. Then open a pull request from pre-launch to main per HGM-SOP-WEB-002, merge it, and confirm the `netlify.app` production build succeeds.

👁 Expected result

The production site on `netlify.app` carries every change from phases 2 to 7. This is what the client sees in phase 9.

Phase 9 — Client sign-off

Written approval, on the record, before DNS moves.

9.1 Send the site to the client through the account manager

Post the `netlify.app` address and the message below for the account manager to forward. **[NEEDS INPUT: Q27 — the channel: email via the account manager, or another]**



MESSAGE FOR THE ACCOUNT MANAGER TO FORWARD

<client>'s new site is ready for your review: <netlify.app URL>. It has been through search, accessibility, performance, device and payment checks, and a test booking has been made and cancelled. Please look at every page, on your phone as well as your computer, and reply with anything you want changed. Once you reply "approved", we will schedule the switch to <client-domain>.

👁 Expected result

The client has the link and knows what "approved" triggers.

9.2 Work through the client's comments

Each change goes through the WEB-002 branch-and-preview path. Re-run the check from the phase it touches — a wording change re-runs task 2.2; a layout change re-runs task 4.4; anything near checkout re-runs phase 7.

👁 Expected result

No comment is open.

9.3 Record the approval and agree the cutover moment

Paste the client's written approval into the Asana task. Agree with the account manager the day and time of the cutover, and who on the client side will be reachable during it. **[NEEDS INPUT: Q7 — who decides the moment and what window HGM offers]**

👁 Expected result

The Asana task carries the approval, the cutover time and the client contact for the day. Without all three, phase 10 does not start.

Phase 10 — DNS cutover

The point of no return. Do it with the section 08 checklist open, with a second person watching the first time, and with the client's email records written down before anything changes.

10.1 Find where the domain lives

Open the client's record in hgmportal.com and read the registrar and the DNS host. If it is not recorded, look the nameservers up. **[NEEDS INPUT: Q1 and Q2 — whether the registrar and DNS host are on the portal record, and whether Cloudflare is common]**



TERMINAL /
CLI

REPLACE <CLIENT-DOMAIN>

```
dig +short NS <client-domain>
dig +short A <client-domain>
dig +short MX <client-domain>
```



CLAUDE
PROMPT

PASTE INTO CLAUDE CODE

Run dig for NS, A, MX and TXT on <client-domain>. Tell me who hosts the DNS, where the site currently points, and whether the domain carries email. Make no changes.



Image to be added

(image to be added) — The registrar or Cloudflare account page showing which nameservers the domain is using. Ring the nameserver list.

👁 Expected result

You know the registrar, the DNS host, and whether the domain carries email.

10.2 Write down every existing record

In the DNS host's record editor, export or screenshot every record — A, AAAA, CNAME, MX, TXT, SRV — and save the capture to the client's 1Password vault with today's date.

Screenshot 10.2 to capture

The DNS host's record editor showing the full record list before any change. Crop to the table; ring the MX rows. Capture a registrar variant and a Cloudflare variant if both turn up.

The MX and TXT records are the client's email

Whatever method task 10.7 uses, those records must exist unchanged afterwards. Nameserver delegation to Netlify DNS removes them unless they are recreated first.

Image to be added

(image to be added) — The full DNS record list before any change — this is the one you will need if you have to roll back.

Expected result

A complete record of the domain as it was, so the rollback in task 10.11 and the email check in task 10.10 have something to compare against.

10.3 Get DNS access, or a person who has it

[NEEDS INPUT: Q3 — whether HGM gets a login to the DNS host, or the client or their IT makes the change while HGM watches]

Expected result

Either you can edit the records yourself, or a named person is booked for the cutover window.

10.4 Lower the TTL

[NEEDS INPUT: Q6 — whether the TTL is lowered in advance, to what, and how long before]

👁 Expected result

The A and CNAME records show the lowered TTL, and the old TTL has had time to expire before task 10.7.

10.5 Add the custom domain in Netlify and switch the site's own address

In the Netlify project, open **Domain management** and add `<client-domain>` and `www.<client-domain>`. Set which one is primary. Then change `NEXT_PUBLIC_SITE_URL` on Netlify to the real domain and re-register the Stripe and booking platform webhooks against it. [\[NEEDS INPUT: Q10 — apex or www as primary\]](#) [\[NEEDS INPUT: Q28 — whether the site URL changes before or after the records change\]](#) [\[NEEDS INPUT: Q15 — the webhook re-registration\]](#)

📄 Screenshot 10.5 to capture

Netlify Domain management with both domains added and the awaiting-DNS state. Crop to the domain list; ring the add-domain button.

📄 Image to be added

(image to be added) — Netlify **Domain management** after the custom domain is added, showing the records Netlify asks for. Ring the record values.

👁 Expected result

Netlify shows both domains awaiting DNS and tells you the records it wants.

10.6 Add the redirect map for old URLs

[\[NEEDS INPUT: Q9 — whether a redirect map exists, who writes it, and whether it is a `\_redirects` file or `netlify.toml` in the template\]](#)

TERMINAL /
CLI

EXPECT A 301 AND A LOCATION HEADER

```
curl -sI https://<site-name>.netlify.app/<old-path> | head -3
```

👁 Expected result

Every URL from the old site's sitemap that has no page on the new site redirects to the right one. Test three.

10.7 Change the records at the agreed moment

[NEEDS INPUT: Q4 — the method: A or ALIAS for the apex and a CNAME for www pointing at Netlify, or delegating the nameservers to Netlify DNS. The task text is written once the method is chosen.] Post in the client websites space that the cutover has started.

 POST IN THE CLIENT WEBSITES SPACE

<client> cutover started at <time>. Old records saved to 1Password. Will confirm when live.

Expected result

The records are saved in the DNS host and the change is announced.

10.8 Watch it resolve

Repeat until the answers change. Propagation takes from minutes up to the old TTL.

 TERMINAL / CLI

REPEAT EVERY FEW MINUTES

```
dig +short A <client-domain>
dig +short CNAME www.<client-domain>
curl -sI https://<client-domain>/ | head -1
```

 CLAUDE PROMPT

PASTE INTO CLAUDE CODE

Every 60 seconds for up to 30 minutes, run `dig +short A <client-domain>` and `curl -sI https://<client-domain>/`. Tell me the moment the A record changes and the moment curl returns 200.

Image to be added

(image to be added) — A DNS lookup showing the new record resolving, from a machine outside the office network.

Expected result

The A record answers with Netlify's address and curl returns HTTP/2 200.

10.9 Confirm the certificate

In Netlify **Domain management**, check the HTTPS section shows a certificate issued for both domains. [\[NEEDS INPUT: Q10 — the typical wait and whether it has ever needed a manual renew\]](#)

Screenshot 10.9 to capture

Netlify's certificate status after cutover. Crop to the HTTPS panel; ring the issued state.

Image to be added

(image to be added) — Netlify's domain panel showing the certificate issued for the apex and www. Ring the certificate status.

Expected result

https://<client-domain> loads with no browser warning.

10.10 Prove the client's email survived

Send a message from your HGM address to the client's address on the domain and ask the client contact from task 9.3 to confirm they received it. Check the MX and TXT records match the capture from task 10.2.

 **TERMINAL /**
CLI

COMPARE WITH THE TASK 10.2 CAPTURE

```
dig +short MX <client-domain>
dig +short TXT <client-domain>
```

Image to be added

(image to be added) — An MX lookup for the domain after the cutover, showing the client's mail records unchanged.

Expected result

The client confirms receipt and the mail records are unchanged. This is rule 2, and it is not optional.

10.11 Rollback, if the site is wrong

[NEEDS INPUT: Q8 — the rollback: restore the saved records to point at the old host, and whether the old site is kept running and for how long]

👁 Expected result

You know, before you need it, how to put the old site back within one TTL.

Phase 11 — Confirm live and hand to WEB-003

Everything that was checked on `netlify.app` is checked again on the live domain, and the repository is proven to be the live site.

11.1 Prove the repository is the live site

This is WEB-003 task 1.1, run for the first time.

TERMINAL /
CLI

REPLACE <CLIENT-DOMAIN> AND <SITE-NAME>

```
dig +short NS <client-domain>; dig +short A <client-domain>
curl -s https://<client-domain>/ > /tmp/live.html
curl -s https://<site-name>.netlify.app/ > /tmp/host.html
diff -q /tmp/live.html /tmp/host.html
git fetch origin && git rev-parse origin/main
```

CLAUDE
PROMPT

PASTE INTO CLAUDE CODE

Prove the repo is the live site: run dig for NS and A on <client-domain>; fetch `https://<client-domain>/` and the `netlify.app` address and compare the HTML; ask the Netlify connector for `get-deploy-for-site` and compare `commit_ref` with `origin/main` after a fetch. Make no changes. Tell me clearly if any of the three do not match.

👁 Expected result

`diff` is silent and the deployed commit equals `origin/main`.

11.2 Repeat the checks that depend on the address

On `https://<client-domain>`: the Open Graph tags (task 2.2) show the real domain, `llms.txt` loads, the Stripe test-mode webhook shows a delivered event from a checkout started on the live domain, and the pixel is in the live HTML.



TERMINAL /
CLI

```
curl -s https://<client-domain>/ | grep -o '<meta property="og:url" content="[^"]*"'
curl -sI https://<client-domain>/llms.txt | head -1
curl -s https://<client-domain>/ | grep -c "fbq('init'"
```

Expected result

Every one passes on the real domain, not on netlify.app.

11.3 Switch Stripe to live

[NEEDS INPUT: Q11 — who swaps the test keys for live keys, where, and whether HGM ever sees the live secret key] [NEEDS INPUT: Q12 — who removes the 100% test coupon]

Expected result

The site takes real payments and the test coupon from task 6.2 is gone.

11.4 Close out and hand to WEB-003

Append the session's row to `TOKENS.md`, save the final `.env.local` to `1Password`, post the live URL in the client websites space, comment on the Asana task with the live URL and the cutover time, and move the task to the next stage. [NEEDS INPUT: Q28 — the Asana stage after pre-launch] Continue in HGM-SOP-WEB-003 the same day.



POST IN THE CLIENT WEBSITES
SPACE

<client> is live at https://<client-domain> as of <time>. Email confirmed arriving. Post-launch connections (WEB-003) start <date>.

Expected result

Somebody else could start WEB-003 tomorrow from the Asana task alone.

The client cannot supply Stripe test keys

Ask for sandbox details instead. If neither arrives, phases 7 and 8 cannot run; do not substitute another client's keys or a Stripe account of HGM's. Raise it with the account manager and Gillian before the cutover date is agreed.

The Lighthouse loop will not reach 100 on one page

Stop the loop after [\[NEEDS INPUT: Q21\]](#) and fix the named items by hand. A third-party embed — the GoHighLevel iframe, a map — can cap a score; record the reason on the Asana task and move on.

The client's domain is on Cloudflare with the proxy on

[\[NEEDS INPUT: Q2 — whether HGM turns the proxy off for the Netlify records or leaves it on. Ridge & Falls was on Cloudflare with no HGM login.\]](#)

The client wants a launch date before sign-off has arrived

Rule 6. The cutover moves; the sign-off does not get skipped.

The old host also serves the client's email

Then the nameservers cannot move without the mail records being recreated first. [\[NEEDS INPUT: Q5 — whether this has come up before\]](#)

The client is on Hostaway, not Guesty

[\[NEEDS INPUT: Q17 — the Hostaway booking engine and webhook equivalents. AwayFrames is the first Hostaway run; record them there.\]](#)

SYMPTOM	CAUSE	WHAT TO DO
Claude reports 100/100 but DevTools shows less	The loop measured localhost or a different build	Run the loop against the same build you are checking (task 4.1)
Resend shows the confirmation as failed	Sending domain not verified, or the key belongs to another domain	Check the domain Resend has verified against the from-address the site uses
Stripe shows the payment, no webhook delivery	Endpoint registered against the wrong address, or the signing secret is wrong	Task 6.8; after cutover, task 10.5
The GHL contact never appears	Wrong Location ID	Compare against the client's record in hgmportal.com (WEB-004 task 7.4)
The site loads on netlify.app but not on the domain after an hour	Records not saved, or the old TTL has not expired	dig from a second network; check the record editor; wait one old TTL
Browser shows a certificate warning on the domain	Certificate not yet issued	Task 10.9; if still missing after [NEEDS INPUT: Q10] , escalate
The client's email stops arriving	MX or TXT records lost in the cutover	Restore them from the task 10.2 capture now, then tell the account manager
Old links from Google land on a 404	Redirect map missing an entry	Task 10.6

Escalate on a failed cutover or the client's email

[NEEDS INPUT: Q29
— **person and channel]**

Escalate on the money path or a payment fault

[NEEDS INPUT: Q29
— **person and channel]**

Escalate on the template's webhooks or variable names

Leshan Patterson, template owner, in the **client websites** space in Google Chat.

Escalate on scope, the deadline or the client

Gillian, Operations Manager, in Google Chat — through the account manager for anything the client has to do.

The cheap moment to say it

A missing key is visible in task 1.1. A cutover with no DNS access is visible in task 10.3. Raise each on the day you see it, not on launch day.

Do not start task 10.7 until every line is true.

- ☐ Phases 2 to 8 complete on the `netlify.app` production build, and the pull request from pre-launch merged.
- ☐ Every audit checked by hand, not only reported by Claude: tags (2.2), schema (2.4), keyboard and alt (3.3), Lighthouse (4.2), a real phone (4.4).
- ☐ Privacy policy and terms of use reachable from the footer; the cancellation policy identical on property page, confirmation and email.
- ☐ Every variable in `ENV.template` set on Netlify; `.env.local` saved to the client's 1Password vault; no key beginning `sk_live_` on any HGM machine.
- ☐ Webhooks registered for Stripe and the booking platform, and a delivered event seen for the test payment.
- ☐ Test booking made, confirmation email seen delivered in Resend, reservation seen in the booking platform and cancelled, contact seen in GoHighLevel.
- ☐ Client's written approval pasted into the Asana task, with the cutover time and the client contact for the day.
- ☐ Registrar and DNS host known; every existing record captured to 1Password; MX and TXT records listed.
- ☐ DNS access confirmed, or the person making the change booked for the window.
- ☐ TTL lowered and the old TTL expired.
- ☐ Both domains added in Netlify Domain management; redirect map in place and three redirects tested.
- ☐ Rollback written down: which records to restore, and the old host still running.
- ☐ A second person watching, if this is your first cutover.

Next: post-launch connections

HGM-SOP-WEB-003 — Google Analytics 4, Search Console, Hotjar, the structured-data check and the ownership map. Starts the day the site is live, from task 11.4.

Before this: the build

HGM-SOP-WEB-004 — the site this SOP starts from, complete on `netlify.app` at its task 8.4.

This SOP is the last of three before launch. It starts where the build ends and stops where post-launch begins.

DOCUMENT	COVERS	WHEN
HGM-SOP-WEB-004 — rebuild a live client site	The build, to a finished site on <code>netlify.app</code>	Before this document; its task 8.4 is this document's task 1.1
HGM-SOP-WEB-005 — this document	Audits, keys, money path, test booking, sign-off, DNS cutover	Between the merged build and the live domain
HGM-SOP-WEB-003 — post-launch connections	GA4, Search Console, Hotjar, structured data, the ownership map	The day the site is live, from task 11.4
HGM-SOP-WEB-002 — Asana to live	Branches, pull requests, deploy previews, merging	Referenced from tasks 1.4, 8.6 and 9.2
HGM-SOP-WEB-001 — env files in 1Password	Storing and updating the client's <code>.env</code> file	Referenced from tasks 1.3, 6.10 and 11.4
The promo-code campaign links document	Coupons and campaign links in the money path	[NEEDS INPUT: to be folded into tasks 7.3 and 7.4]
The WCAG 2.2 AA audit prompt	The criteria for phase 3	[NEEDS INPUT: to be folded into task 3.4]

 The site is not launched when DNS resolves

It is launched when the booking flow, the payment, the email and the client's own inbox have all been seen working on the real domain. Task 11.2 is the launch; task 10.7 is just the switch.

10 REVISION HISTORY

DATE	VERSION	AUTHOR	CHANGE
2026-09-29	0.1	Kyle Zinger	Field-test draft from the notebook's Launch Process pages and the kickoff prompt. Interview answers, the WCAG audit prompt and the promo-code document outstanding; first run is AwayFrames.
2026-10-06	0.2	Kyle Zinger	Nineteen image placeholders added, weighted to the keys and cutover phases, ahead of the portal upload.

REMEMBER

"Nothing is verified until you have seen it on the live domain, and nothing that touches money is done outside Fable against a test key."

Next

HGM-SOP-WEB-003 — post-launch connections